

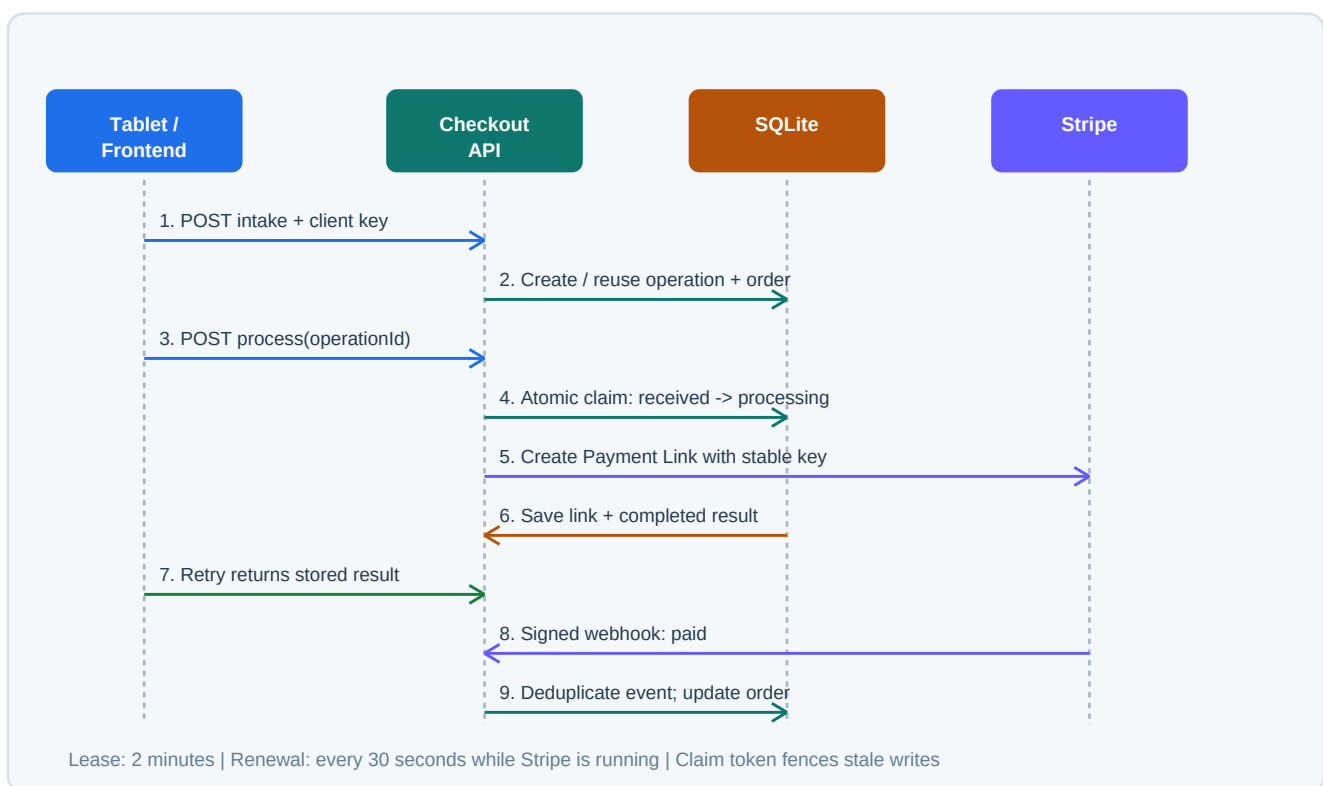
Hair-Expo Payment Idempotency

How the checkout flow prevents duplicate operations, orders, and Payment Links

Purpose

This note explains how Hair-Expo gives one customer checkout one stable identity. If the user clicks again, the network retries, or Stripe sends the same event again, the system can recognize the same work and reuse the existing result.

Core rule: one checkout intention keeps the same identity from the frontend to SQLite and then to Stripe.



At a glance

Frontend	Creates one client idempotency key and keeps it for the current checkout.
Checkout API	Creates or reuses one checkout operation, claims it once, and returns the saved result on retry.
SQLite	Stores the operation, order, attempt, payment references, claim token, webhook event IDs, and final response.
Stripe	Receives the same provider idempotency key, so a retry does not create another Payment Link.

1. The identity chain

Each layer keeps the same checkout identity. This makes retries safe and easy to recognize.

Layer	What is stored or reused	Why it matters
Client	Client idempotency key	Repeated intake requests mean the same customer action.
Operation	checkout operation ID	The backend has one record for the checkout workflow.
Order	order ID and order number	A retry reuses the same local order.
Attempt	one checkout attempt and provider key	The provider call has one stable identity.
Provider	Stripe idempotency key	Stripe returns the original result for the same request.

2. Intake: one customer action, one operation

The frontend sends the cart, customer data, and a client idempotency key to the intake API. The backend hashes the request and stores the key with the operation. A repeated request with the same key and same data returns the existing operation ID. The same key with different data is rejected as a conflict.

Example

Same key + same cart -> return the same operation ID.

Same key + different cart -> reject with a conflict. This prevents accidental reuse of a key for a new checkout.

3. Atomic claim: only one worker proceeds

When the frontend calls the process API, SQLite changes the operation state in one transaction. The update succeeds only when the operation is eligible, such as received or a safely recoverable state. Two requests arriving together cannot both win the claim. The first request gets the current claim token; the other sees that the operation is already owned.

Why the claim is important

The provider call happens outside the SQLite transaction. The atomic claim closes the local race before the system talks to Stripe.

4. Lease and fencing token

A processing claim is temporary. Hair-Expo gives it a 2-minute lease. While the Stripe request is running, the checkout core renews the lease every 30 seconds. A live request therefore is not reclaimed just because Stripe is slow.

2 minutes	Maximum time before an abandoned claim may be recovered.	30 seconds	Renewal interval while the provider request is active.
Claim token	A unique token is written when a worker claims the operation.	Fencing	A stale worker cannot save a late result because its token no longer matches.

If the process crashes, renewal stops. After the lease expires, a later request may recover the operation. It receives a new claim token. An old request that returns late is fenced out by conditional database updates.

5. Stable Stripe idempotency

The provider request uses a stable key derived from the checkout operation, for example **trunov:payment-link:<operationId>**. Every retry for that operation uses the same key. If the first Stripe response is lost, the retry is still the same logical request. Stripe can return the original Payment Link instead of creating another one.

Important boundary

The local claim prevents two normal workers from proceeding at the same time. The Stripe idempotency key protects the external call if a timeout or crash creates uncertainty.

6. Stored result reuse

After the provider succeeds, Hair-Expo stores the checkout URL, provider references, Payment Link ID, order ID, order number, and pricing result. A later request for a completed operation returns that stored response. It does not create a new order or call Stripe again.

7. Webhooks and payment status

Creating a Payment Link and receiving payment are separate events. Stripe sends a signed webhook when payment succeeds. Hair-Expo stores the event ID and ignores a webhook that has already been processed. The order changes to paid only through a valid, conditional state transition.

Event	Local action
Signed payment webhook arrives	Verify signature and identify the order.
Event ID is new	Store the event ID and process the payment update.
Event ID already exists	Treat it as a duplicate and do nothing again.
First valid payment	Mark the order paid and deactivate the Payment Link.

8. Conditional state transitions

Database updates include the expected current status and, when needed, the claim token. For example, a provider result is saved only if the operation is still `payment_pending` and the token still belongs to the current worker. If no row changes, the system does not silently report success. It reloads the current state or returns a conflict.

Result

A stale request cannot overwrite a newer result. A duplicate completed request returns the saved result. A duplicate webhook does not mark the order paid a second time.

9. Payment Link deactivation

After the first valid payment, the backend deactivates the Stripe Payment Link and records the payment references. This reduces the risk that the same link is used for a second payment. The local order transition remains conditional, so an already-paid order is not changed incorrectly by a repeated event.

10. SQLite deployment requirement

This design uses SQLite for a small booth workflow. It is suitable when the backend runs as one instance with a persistent disk. The database file and its WAL and SHM files must survive restarts and redeployments.

Required	Reason
One backend instance	Avoid two servers writing to separate local SQLite files or racing over unsupported shared storage.
Persistent disk	Keep operations, orders, attempts, event IDs, and provider references after restart.
Preserve SQLite side files	Keep the main database, WAL, and SHM files together.
Do not use ephemeral serverless storage	A temporary filesystem can lose the database or create inconsistent state.

11. Failure examples

Situation	What Hair-Expo does
User double-clicks	Same client key and operation. The second request cannot claim the operation again.
Network response is lost	Retry uses the same operation and same Stripe idempotency key. The stored or provider result is reused.
Backend crashes during provider call	Lease eventually expires. A later worker gets a new claim token. The old worker cannot save over it.
Stripe sends webhook twice	The stored event ID makes the second webhook a no-op.
Payment is completed	Order becomes paid and the Payment Link is deactivated.

Summary

Hair-Expo uses one identity chain: client key -> operation ID -> order and attempt -> stable Stripe key. Atomic claims, renewable leases, fencing tokens, conditional updates, stored results, webhook deduplication, and Payment Link deactivation work together to make retries safe.

Source basis

Prepared from the latest visible Hair-Expo repository state and the referenced commits:

58c2899e8342709b4b6b977604d634ed1c19bdd2 - lease renewal failure logging; a13624d0a6ba782f8e81f534386c1e0ca745d827 - provider retry fencing and lease renewal; de0ba6d6b8533bc37f547d73e5ae11bf16a80aa3 - checkout workflow and payment hardening.